Software Requirements Specification

for

OpenBench

A policy-pluggable internal marketplace for people, exclusive resources, and metered capacity

Version 0.4

2026-11-07

Prepared by

Jai Nana Brian Lujan

Fotih Turopov Yerassyl Shnazbayev

CSDS 393 Software Engineering

Case Western Reserve University

Revision History

Name	Date	Reason for changes	Version
J. Nana, B. Lujan, F. Turopov, Y. Shnazbayev	2026-09-19	Initial draft assembled from the design decisions log (D1–D17).	0.1
J. Nana	2026-09-21	Sections 1–6 reworded after team review and the feature descriptions condensed; references revised; the Stretch requirements added; business rules, written-out use cases, and the usability test record moved to the full edition.	0.2
J. Nana	2026-10-05	Overlapping-Loan rule made an eligibility rule (one Loan at a time) and Shortlist order changed, after grader feedback; inspection participation made explicit; test database changed from an in-memory database to PostgreSQL in a container to match the design.	0.3
J. Nana	2026-11-07	An Optional Matched line still awaiting approval no longer delays allocation and is no longer dropped at it; its Loan forms when approved (Lifecycle.Evaluate, Lifecycle.Allocate, Matched.Loan.Form); the Objection Window and the Shortlist start at submission; the Refund lead time is measured to the start of the released part's occurrence.	0.4

How to read this document. Section 1 is the Vision and Scope statement; Sections 2–6 are the specification itself, and the appendices hold the glossary, the data and analysis models, the use cases, the inspection report, and the traceability table. Functional requirements carry dot-notation identifiers such as **Draft.Submit.Lock**; use cases UC-n are listed in Appendix D. Capitalized words such as Request, Hold, and Loan are defined in the glossary, Appendix A, which is generated from the team's shared vocabulary file. A longer edition of this specification with the complete requirement set, the business rules, and the use cases written out in full is kept in the project repository; identifiers are the same in both.

Contents

1 Vision and Scope **2**

2 Introduction **4**

3 Overall Description **5**

4 System Features **7**

5 External Interface Requirements **13**

6 Other Nonfunctional Requirements **14**

A Glossary **15**

B Data Dictionary and Data Model **17**

C Analysis Models **19**

D Use Cases **20**

E Inspection Report **21**

F Traceability **21**

1 Vision and Scope

1.1 Background, Business Opportunity, and Customer Needs

Companies hold several types of scarce internal capacity: their people's time, rooms and lab equipment, GPU hours, and license seats. Much of it goes unused because no one can see it. Managers keep engineers on the books between projects, rooms are reserved for long periods but never used, and the shared GPU pool creates queues where the loudest Team prevails. Every existing solution deals with only one aspect of this problem: calendaring systems schedule manually, talent marketplaces connect people, and cluster schedulers ration compute. There is no central system where a Manager can ask for "two engineers with PyTorch experience for six weeks, four GPUs for two of these weeks, and a lab room on Thursdays" and receive a single response. Our proposal cites two internal talent marketplaces as evidence that making capacity visible recovers much of it: Unilever redeployed over 8,000 employees and 300,000 hours, and Schneider Electric identified about 127,000 idle hours within weeks of launch.

1.2 Business Objectives and Success Criteria

BO-1 Enhance usage of shared Exclusive and Metered Resources by highlighting their availability and making requests easy in one platform.

BO-2 Make lending a person worthwhile for the Manager, who will then let go of the person between projects rather than retain them.

BO-3 Replace the piecemeal negotiating over capacity with a single process that is time-bound and auditable, with results visible to all parties involved.

SC-1 The final demonstration takes a Request spanning all three Allocation Classes to Allocated and shows its Charges and Earnings in both Teams' Statements.

SC-2 From the first Period of use, the dashboard shows how much of each Resource Type was used or sat idle and how many credits each Team earned and spent.

SC-3 No Request waits on an unresponsive Manager beyond the Objection Window, and every Request finishes in Allocated, Waiting, Refused, or Withdrawn with its complete history kept.

SC-4 A Manager with no training can make and submit a three-line Request within five minutes, tested with at least three people from outside our team.

1.3 Vision Statement

For Managers seeking people, rooms, and compute controlled by another Team, OpenBench serves as an internal marketplace. The Manager makes one Request covering all three categories, and each part is allocated under the policy that fits it: people by skill and their own Manager's approval; rooms and instruments with conflict detection, a waitlist, and priority bumping; metered pools with capacity and quota checks. A booking calendar, a talent marketplace, and a cluster scheduler each cover one category. OpenBench covers all three within one Request and binds them together through a Ledger of credits, where lending a person earns the credits that a room or a GPU allocation costs. Credits are for showback only; no money changes hands.

1.4 Major Features

FE-1 Inventory and Resource Types as data

FE-2 Member profiles, skills and availability

FE-3 Suggestions from Members to their Manager

FE-4 Draft and submit a Request with a locked Quote

FE-5 Matched allocation, Shortlist and one approval

FE-6 Exclusive allocation, conflicts, waitlist, Bump

FE-7 Metered allocation, Pool Capacity and Quota

FE-8 Request lifecycle with bounded Holds

FE-9 Release with a partial Refund

FE-10 Ledger and pricing, Grants, Charges, Earnings, Price Bands

FE-11 Showback dashboard and Statement

FE-12 Administration and Capacity Planner settings

1.5 Scope of Initial and Subsequent Releases

Release 1 is what we promise to deliver this semester. Stretch features are Low-priority requirements in Section 4, built only if time allows. Future features are named so the design is prepared for them; they have no requirements.

Area	Release 1	Stretch	Future
Request	One Request including all three classes. Required and Optional lines. Suggestions. Dashboard and Statement	—	Demand analytics on declined Suggestions
Matched	Skill filter, availability, Shortlist, one approval by the lending Manager	—	Person consent. Skill-based Day Rate
Exclusive	Conflict detection, waitlist, Priority lines that Bump	—	Team rank set by the Capacity Planner
Metered	Pool Capacity check. Per-Team Quota	Service tiers and preemption (Priority lines on Metered pools)	—
Ledger and pricing	Grant, Charge, Earning, Refund. Grants expire at Period end, Earnings carry over. Refund Schedule. Unit Price, Price Bands, priority premium. Locked Quote	Actual usage recording. Requested and actual usage comparison. Demand surcharge	Inter-Team transfers. Forecasting accuracy reward. Usage feeds. Auctions
Platform	Username and password. In-app notifications. Web browser only. No integrations	Email notifications	SSO; calendar, HR, and cluster-scheduler integration

1.6 Limitations and Exclusions

LI-1 One company, one time zone, one deployment. Desktop web browser only, with no mobile layout and no high-availability configuration.

LI-2 No real money transactions occur. Credits exist for showback only, so there are no invoices and no finance system.

LI-3 No external system integrations in Release 1: no calendar, HR, identity, or cluster scheduler.

LI-4 A Person is informed of the Loan but cannot refuse it in Release 1.

1.7 Stakeholder Profiles

Stakeholder	Major value	Attitudes, interests, constraints
Management	Idle capacity back in productive use, less hoarding, and visibility of who uses what and who wants what	Wants showback information without a billing process, and will not pay for chargeback.
Managers	A single point to request people, rooms, and compute, and a reply within a certain timeframe	Concerned about losing their people. Want Priority left open for emergencies. Have little time to learn a new tool.
Members (as Persons)	To be able to see their own skills and Loans and to make Suggestions to their Manager	Would like to be consulted before being loaned. Cannot decline a Loan in Release 1.
Resource Owners	Correct Inventory information and no approval chores	Would like allocations to account for changes in capacity. Manage only their own Resources.
Capacity Planner	Grants, prices, and Refund Schedules to shape demand	Would like to have few escalations, and those solvable with a Grant. Sets policies but does not administer users.
Administrator	Ability to create Resource Types without coding; simple role management	Does not want to make policy decisions. Configures the system but does not set prices.

1.8 Business Risks

Each risk gives its probability, then its impact: low, medium, or high.

RI-1 Managers don't lend, so the marketplace is always empty (medium, high). Mitigation: Matched lines cost the borrower nothing, every Loan earns the full Day Rate, and earned credits never expire.

RI-2 Holds tie up rooms for Requests that end up not going ahead (medium, medium). Mitigation: Holds never survive the Objection Window, and the waitlist fills the freed room.

RI-3 Incorrect Grants or price settings can result in hoarding or wasted capacity (medium, medium). Mitigation: Price Bands, mid-Period Grant adjustments, and the dashboard's idle-capacity view.

RI-4 Managers over-reserve rooms and Release them late (medium, medium). Mitigation: the Refund Schedule never refunds 100% and drops sharply inside seven days of the booked window's start.

RI-5 Four students, one semester (high, high). Mitigation: Release 1 is fixed; Stretch and Future items are plug-ins behind the same allocation interface.

2 Introduction

2.1 Purpose

This Software Requirements Specification defines the functional and non-functional requirements of Release 1 of OpenBench. OpenBench is an internal marketplace in which the Manager of a Team places one Request for people, Exclusive resources, and Metered capacity. Each portion is assigned according to the allocation policy that applies to it, and the outcome is settled in credits. All requirements are committed to Release 1 unless they are tagged Stretch. The specification is intended for the development team, the course staff who grade and inspect it, and the stakeholders of Section 1.

2.2 Project Scope and Product Features

Release 1 offers all twelve major features, FE-1 to FE-12, enumerated in Section 1: Resource Types as data and the Inventory; Member profiles; Suggestions; creation and submission of a Request with a locked Quote; the three allocators (Matched, Exclusive, and Metered) behind one interface; the Request lifecycle, including Holds that never outlive the Objection Window; Release with a partial Refund; the Ledger with Grants, Charges, Earnings, Refunds, and Price Bands; the showback dashboard and the Statement; and administration. The Vision and Scope section states why some elements are deferred to Stretch or Future.

2.3 Document Conventions

- **Shall** denotes a mandatory requirement. Priority is *High* (Release 1 cannot be used without it), *Medium* (Release 1 is usable without it but not complete), and *Low* (Stretch).
- Functional requirements all have an identifier that starts with their feature name using dot notation, e.g., **Draft.Submit.Lock**. Use cases start with UC- and features with FE-. Other prefixes are documented on their first appearance. Identifiers are shared with the full edition of this specification, which holds the complete requirement set, the business rules, and the use cases written out in full.
- Terms such as Request, Hold, Loan, and Objection Window that are capitalized are defined in the glossary, Appendix A. We used the language a Manager uses in conversation and dropped financial language. Credit means the unit of account and never a direction of entry, and nothing in the system is paid, funded, or billed.
- Values marked *example default* are settings of the Capacity Planner. The listed value is the one that will be shipped and tested, and it can be changed by the Capacity Planner without a code change.

2.4 References

1. J. Nana, B. Lujan, F. Turopov, *OpenBench: A Policy-Pluggable Internal Resource Marketplace*, CSDS 393 informal project proposal, 2026.
2. CSDS 393, *Rubric for Software Requirements Specification*, 2026.
3. Project vocabulary file `CONTEXT.md`, decisions log, and architecture decision records, project repository, 2026.

2.5 Definitions

Appendix A lists every defined term and is generated from the project's vocabulary file at build time. A first-time reader needs only a few of them. A *Team* has one *Manager* and one *Account* of credits. A *Request* is a set of *Request Lines*, and each line belongs to exactly one *Allocation Class*: *Matched* for people, *Exclusive* for things held whole over a window, or *Metered* for divisible pools. A Matched line that has been satisfied is a *Loan*. A *Hold* is a tentative allocation that is not yet charged and lasts no longer than the *Objection Window*. The *Ledger* records four kinds of *Transaction*: *Grant*, *Charge*, *Earning*, and *Refund*.

3 Overall Description

3.1 Product Perspective

OpenBench is a new, standalone web application. It replaces the combination of booking calendars, spreadsheets, chat messages, and negotiations that Teams presently employ in order to obtain people, rooms, and computing from each other. Release 1 has no interface with external systems (LI-3). Consequently, all flows in Figure 1 occur between the system and its five actors. There is a single server program serving the web browser front end and the REST API. The database behind it contains the Inventory, the Requests, and the Ledger.

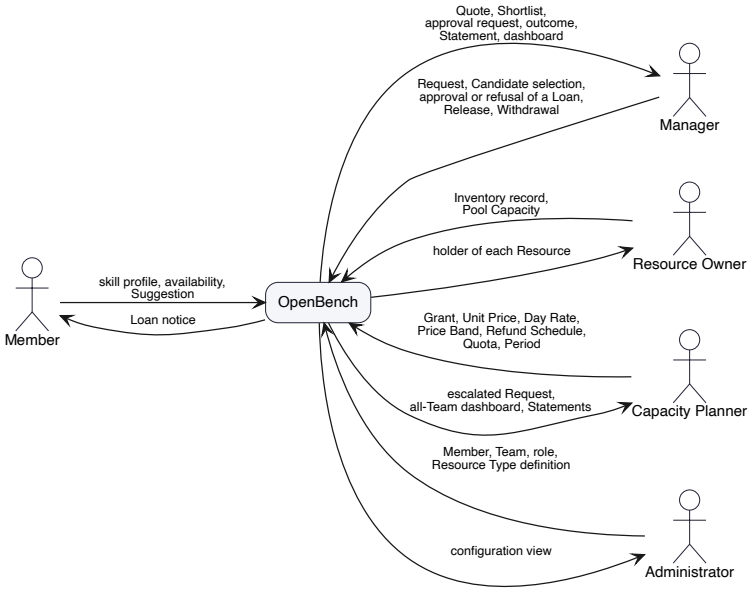


Figure 1: Context diagram. The five actors are roles held by Members; one Member may hold several.

3.2 User Classes and Characteristics

Everyone becomes a *Member* in the system. The Administrator has the ability to give another four types of roles to a Member, and one Member could have many of them at once, for example a Manager who is also the Capacity Planner. The server checks a Member's roles on every action (SE-1).

User class	Characteristics and what they do in the system	Importance
Member	Up to 500 people with varying levels of technical skill who access the system several times a month. A Member updates their technical skills and availability information, makes Suggestions to their Manager, and views their Loans. Any Member can be loaned out as a Person.	Favored (as Person)
Manager	Up to 50 people, one for each Team, who access the system on a weekly basis. The Manager is the sole actor who drafts and submits Requests, selects Candidates, approves or refuses Loans of their own Team's Members, Releases lines, and views their Team's Statement.	Most important
Resource Owner	Dozens of people who update the Inventory records of specific Exclusive and Metered Resources, including Pool Capacity. A Resource Owner does not approve any allocation.	Important
Capacity Planner	1 or 2 people. The Capacity Planner decides on each Team's Grant per Period, Unit Prices, Price Bands, the Day Rate, Refund Schedules, Quotas, the priority premium, the bump cutoff, the Objection Window, and the Period. Escalations come to the Capacity Planner, who decides a lapsed approval on the silent Manager's behalf or settles the matter by adjusting the Grant mid-Period. They also view the dashboard and Statements for every Team.	Most important
Administrator	1–2 IT people. The Administrator manages Members and Teams, assigns roles, manages the skills list, and specifies Resource Types as data. Policy is not set by the Administrator.	Important

3.3 Operating Environment

- **Server:** one on-site server with one Java 17 (Spring Boot) process, packed into a Docker image. The process handles the REST API and serves the compiled front end as static files. An nginx reverse proxy is placed in front of the server if TLS termination is required.
- **Database:** PostgreSQL 16 for production use, and PostgreSQL 16 in a container started by the test suite (Testcontainers) for test automation, with the same schema migrations.
- **Clients:** latest versions of Chrome, Firefox, Safari, and Edge. No mobile view is present.
- **Organization:** one firm in one time zone. All windows, Price Bands, and Periods are expressed in that time zone.

3.4 Design and Implementation Constraints

CO-1 The back end shall be implemented using Java 17 and Spring Boot and shall be packaged using Maven. The front end shall be implemented using TypeScript, React, and Vite, and the same Maven packaging shall include it in the Spring Boot application.

CO-2 Persistent data shall be stored in PostgreSQL 16, and the tests shall run against PostgreSQL 16 in a container started by the test suite (Testcontainers) with the same schema migrations as production.

CO-3 The front end shall communicate with the back end only via the REST/JSON API. All actions that are available in the user interface shall be available in the API too, with the same role restrictions.

CO-4 Committing Holds and posting the corresponding Charges and Earnings shall happen in one database transaction.

CO-5 Timed functionality (expiration of the Objection Window, Hold and waitlist expiry, Period rollover) shall run as scheduled jobs inside the server process rather than as separate cron jobs.

CO-6 User authentication shall use username and password, and passwords shall be stored hashed. Roles shall be enforced by the Spring Security framework running in the server.

CO-7 Each Allocation Class shall be one allocator behind a common interface, so that a new class can be added by implementing that interface without changing the existing allocators (QA-6).

3.5 User Documentation

UD-1 A brief guide consisting of one page per role, available within the application. This guide explains how to create a Request, understand a Quote and the Available Balance, approve a Loan, Release a line, and view a Statement.

UD-2 An installation and configuration guide covering the Docker image, database configuration, creation of the first Administrator, and the Capacity Planner settings along with their example defaults.

UD-3 Help text for each field of each Capacity Planner setting and each column of the Draft Request screen.

3.6 Assumptions and Dependencies

AS-1 A Capacity Planner exists and assigns a Grant to every Team prior to the first Period. Without Grants, there is no way for a Team to afford an Exclusive or Metered line.

AS-2 Inventory is maintained by Resource Owners, in particular Pool Capacity. Conflicts are checked only against the system's own records.

AS-3 Loans are recorded honestly by Managers. There is no way for the system to confirm that a borrowed Person did work for the borrowing Team.

AS-4 Availability of a Member is determined by their profile information. No calendar is consulted by the system.

DE-1 The system requires Java 17, Spring Boot, PostgreSQL, and current desktop browsers to be available on the company's infrastructure.

DE-2 The build and documentation toolchain of the project is pinned in the repository's Nix flake, so teammates need Nix to reproduce the build.

4 System Features

Each feature gives a short description with its priority, a stimulus/response pair, and the functional requirements that specify it. Identifiers match the full specification, which holds the complete requirement set; this section covers what is needed to understand and test Release 1. Stretch requirements are tagged [Stretch], have Low priority, and are built only if time allows.

4.1 Inventory and Resource Types

Description and priority. The Inventory records every Resource, its Resource Type, and who holds it or has a Hold on it. An Administrator defines Resource Types as data, so a new kind of room, instrument, or pool needs no code change, and Resource Owners keep the Resources current (FE-1; UC-1, UC-2). **Priority:** High. **Stimulus/response.** A Resource Owner reduces a pool's units; the system refuses any reduction below what is allocated or held.

Inventory.Type.Define The system shall let an Administrator define a Resource Type as data: a unique name, exactly one Allocation Class (Matched, Exclusive, or Metered), and named attributes of type text, number, date, or list of allowed values. Defining, changing, or retiring a type shall require no code or deployment change; a retired type accepts no new lines, a type referenced by any line is never deleted, and the built-in type Person (class Matched) cannot be retired.

Inventory.Resource.Add The system shall let a Resource Owner add, edit, and retire an Exclusive or Metered Resource, filling in the type's attributes. A Metered Resource records the units it supplies; the Pool Capacity of a Metered type is the sum of the units of its unretired Resources. A retirement or reduction that would fall below the units allocated or held in any future hour shall be refused, naming the earliest such hour.

Inventory.View.Holder The system shall show, for any Resource and window, whether it is free, on Hold, or allocated in each part of the window and by which Team and Request, and shall let a Member filter Resources by type, attribute value, and being free for a window. A Resource Owner has no approval step in any allocation.

4.2 Member Profiles

Description and priority. A Member's profile holds their Team, skills, and availability, and Matched lines are matched against it (FE-2). **Priority:** High. **Stimulus/response.** A Member updates their skills or availability; the system rebuilds the Shortlist of any unsatisfied Matched line the Member now qualifies for.

Profile.Skills A Member shall belong to exactly one Team, set by an Administrator, and shall maintain their own skills, each chosen from the skill list an Administrator maintains, so that matching is exact.

Profile.Availability A Member shall record availability as date ranges during which they may be lent; they are available for a window when every day of it lies inside a range. A Loan never changes these ranges, but a Person is in at most one Loan at a time: a Member with a Loan overlapping any day of a Matched line's window is not a Candidate for that line, and a Loan that ends early frees the Person from the day after its new last day. A Member sees their own profile and Loans; a Manager sees those of their Team's Members and may edit a Member's availability. A profile holds no personal data beyond name, email, Team, skills, and availability (SE-3).

4.3 Suggestions

Description and priority. A Suggestion is one Request Line that a Member proposes to their own Manager; it affects nothing until the Manager accepts it into a Request and submits. Declined Suggestions are kept as the data for the Future unmet-demand analytics (FE-3; UC-3). **Priority:** Medium. **Stimulus/response.** A Member submits a Suggestion; the Manager accepts it into a Draft Request or declines it with an optional reason, and the Member is notified.

Suggest.Create The system shall let a Member create a Suggestion addressed to their own Manager containing exactly one proposed line (Resource Type, quantity, window, for a Matched line a skill set or a named Person, optional note). A Suggestion is Open, Accepted, or Declined; only Open ones change state, none is ever deleted, and a Suggestion has no effect on the Inventory, Holds, or the Ledger until the line it was accepted into is submitted.

Suggest.Accept.Existing The system shall let the Manager accept an Open Suggestion into a Draft Request as a new line or into an existing line of the same Resource Type, so that several Suggestions point at one line;

an Accepted Suggestion records the line it became and keeps its original ask unchanged. The Manager may instead decline it with an optional reason, and the Member is notified.

4.4 Draft and Submit a Request

Description and priority. A Manager builds a Request in Draft, marks each line Required or Optional, sees a Quote per line, and submits; submission is refused while the Required lines exceed the Available Balance (FE-4; UC-4). **Priority:** High. **Stimulus/response.** The Manager submits; the system locks every Quote and evaluates the Request (Section 4.8), or shows the shortfall and leaves it in Draft.

Draft.Create Only a Manager shall create, edit, submit, or withdraw a Request, and only on behalf of their own Team. Withdrawing from Draft, Pending, or Waiting releases all Holds and held credits; editing a Waiting Request returns it to Draft, removes its lines from waitlists, and recomputes Quotes on resubmission.

Draft.Line.Add A Request Line shall name a Resource Type, a quantity, and a window; its Allocation Class is taken from the type. A Matched line gives a skill set and quantity or a named Person with quantity one; an Exclusive line may name a specific Resource, a weekly recurrence, and the Priority mark; a Metered line gives units held concurrently. A line whose window does not start in the future, or whose type is retired, shall be refused.

Draft.Line.Required Every line shall be Required (default) or Optional. The Manager may split a line into two of the same type to mark a part Optional; there is no per-line partial-acceptance setting.

Draft.Quote.Line While drafting, the system shall show each Exclusive and Metered line's Quote computed as in Ledger.Price.Formula at the current settings, the sums for Required and Optional lines, the Available Balance, and any shortfall; Matched lines show no Quote because they cost nothing. The system shall refuse submission while the Required sum exceeds the Available Balance.

Draft.Submit.Lock On submission the system shall lock every Quote, record the submission time (used for waitlist order), and evaluate every line as specified in Section 4.8; a locked Quote never changes and the Charge posted equals it.

4.5 Matched Allocation

Description and priority. A Matched line asks for people by skills and quantity, or by name. The system builds a Shortlist that leaves out anyone already on a Loan in the window and lists the least-lent first; the requesting Manager picks Candidates, and each Candidate's own Manager approves within the Objection Window or the Capacity Planner decides. An approved Candidate becomes a Loan when the Request is Allocated; the borrowing Team is charged nothing and the lending Team earns the Day Rate (FE-5; UC-5, UC-6, UC-7). **Priority:** High. **Stimulus/response.** A lending Manager approves or refuses; the last Required approval allocates the Request, and a refusal releases every Hold and asks the requesting Manager for a replacement.

Matched.Shortlist.Build When a Request is submitted, the system shall build for each skill-based Matched line a Shortlist of every Member outside the requesting Team who holds every listed skill, is available for the whole window, and has no Loan overlapping any day of it, ordered by fewest person-days lent in the current Period (the days of every formed Loan that fall in the Period), ties by name; a named Person is checked only for availability and an overlapping Loan. The requesting Manager shall select exactly the line's quantity of Candidates; the system never selects automatically, and a Shortlist shorter than the quantity leaves a Required line unsatisfied and the Request Waiting.

Matched.Approve.Route For each selected Candidate the system shall send one approval request to that Candidate's Manager, stating the requesting Team, the window, and the Earning; the lending Manager approves or refuses with an optional reason, recorded with actor and timestamp. No approval request is sent to, or accepted from, the requesting Manager for their own Team.

Matched.Refuse.Replace When an approval is refused, the system shall ask the requesting Manager to select a replacement; the Objection Window shall not restart. If no Candidate remains, or the line named a Person, a Required line places the Request in Refused and an Optional line is dropped.

Matched.Escalate When the Objection Window lapses with an approval outstanding, the system shall present the outstanding approvals to the Capacity Planner, who approves or refuses on the silent Manager's behalf; every later approval on that Request goes to the Capacity Planner directly.

Matched.Loan.Form When the Request becomes Allocated, each approved Candidate shall become a Loan for

the line's window and an Earning of the Day Rate times the person-days shall post to the lending Team; an Optional line still awaiting approval stays open on the Allocated Request and forms its Loan and Earning when its last approval arrives. The Candidate is notified when selected and the Person when the Loan forms; the Person cannot decline in Release 1.

4.6 Exclusive Allocation

Description and priority. An Exclusive line asks for a Resource held whole over a window. The system checks allocations and Holds for conflicts, picks a free Resource when none is named, and waitlists an unmet Required line. A Priority line carries a premium, can Bump a standard line outside the bump cutoff, and is never Bumped itself (FE-6; UC-8, UC-10). **Priority:** High; Priority and Bump are Medium. **Stimulus/response.** A Priority line finds no free Resource; the system Bumps a standard line, refunds it in full, and moves it to the head of the waitlist.

Exclusive.Conflict An instance shall be free for a window when no allocation and no Hold on it overlaps any part of the window; a line is satisfiable when its quantity of free instances exists for every occurrence on the same instance. When no Resource is named the system picks the free instance with the fewest allocated hours in the Period.

Exclusive.Waitlist.Join A Required Exclusive line that is unsatisfiable shall join the waitlist for its Resource Type and window; an Optional line never does. Waitlists are ordered Priority lines first, then by submission time, with a Bumped line at the head; a waiting line leaves the waitlist when its window start passes and its Request stays Waiting; the Manager sees each waiting line's position. When capacity frees through a Release, a Bump, or an expired Hold, the system offers it to the waiting lines in order and satisfies the first whose window fits, under Lifecycle.Waiting.Reevaluate.

Exclusive.Priority.Bump The Manager shall be able to mark an Exclusive line as a Priority line while drafting, at the priority premium; Priority is offered for Exclusive lines only in Release 1. When a Priority line finds no free instance, the system shall Bump the standard lines, allocated or on Hold, that block the instance needing the fewest Bumps, preferring the latest-submitted, provided no Bumped window or occurrence starts within the bump cutoff (example default 24 hours); a Priority line that cannot Bump joins the waitlist ahead of all standard lines. A Priority line is never Bumped and never Bumps another Priority line.

Exclusive.Bump.Effect When a line is Bumped the system shall, in one transaction, post a Refund equal to its full Charge, place it at the head of its waitlist, notify its Manager with the reason, move its Request to Waiting if the line was Required in an Allocated Request while the other lines stay allocated, and record the Bump with both lines, actor, and timestamp.

4.7 Metered Allocation

Description and priority. A Metered line asks for units held concurrently over a window. It fits when, in every hour, allocated plus held plus requested units stay within the Pool Capacity and within the Team's Quota where one is set; the Quota stops one Team with many credits from taking the whole pool. Service tiers that Bump and requested-versus-used tracking are Stretch (FE-7). **Priority:** High. **Stimulus/response.** A Metered line is submitted; the system checks it hour by hour and holds, allocates, or waitlists it.

Metered.Line.Form A Metered line shall name a Resource Type, units to hold concurrently, and a window; no specific Resource and no Priority mark is offered; the line is satisfied with its full units for its full window or not at all. Its Quote is the sum over hours of units times Unit Price times Price Band multiplier.

Metered.Pool.Check A Metered line shall be satisfiable only if, for every hour of its window, the units allocated plus held for that type plus the line's units do not exceed the Pool Capacity and, where the Capacity Planner has set a Quota for the requesting Team on the type, the Team's allocated plus held plus requested units do not exceed it. A Required line that does not fit joins the waitlist under the rules of Exclusive.Waitlist.Join and is satisfied when its units fit in every hour.

Metered.Tier.Define [Stretch] The system shall let the Capacity Planner define service tiers for a Metered Resource Type, each with a premium multiple on the Unit Price, so that a Manager can mark a Metered line with a tier in the same way as a Priority line.

Metered.Tier.Bump [Stretch] A line of a higher tier shall be able to Bump units of lower-tier lines under the rules of Exclusive.Priority.Bump to Exclusive.Bump.Effect, the bump cutoff included; the Bumped line shall receive a full Refund and the head of the waitlist.

Metered.Usage.Record [Stretch] The system shall let a Resource Owner record, at the end of a Period, the units actually consumed by each Metered allocation of that Period.

Metered.Usage.Compare [Stretch] The dashboard shall show requested versus used units per Team and Resource Type per Period. Recorded usage shall have no effect on the Ledger.

4.8 Request Lifecycle and Holds

Description and priority. Exclusive and Metered lines resolve at once; Matched lines wait for approval. Allocating the room before the approvals arrive charges the Team for a room it may never use, so a Request that enters Pending places a Hold on every Exclusive and Metered line that fits. A Hold blocks capacity, posts no Charge, and never outlives the Objection Window (Figure 3; FE-8). **Priority:** High. **Stimulus/response.** The last Required approval arrives; the system commits every Hold and posts every Charge and Earning in one transaction; Optional approvals still outstanding do not delay it. An approval is refused or the window lapses; the Holds release and the Request moves to Waiting.

Lifecycle.States A Request shall be in exactly one of Draft, Submitted, Pending, Allocated, Waiting, Withdrawn, or Refused, with exactly the transitions of Figure 3; every state change of a Request or line records actor, change, and timestamp.

Lifecycle.Evaluate On submission the system shall, in one transaction, decide for every Exclusive and Metered line whether it is satisfiable now and for every Matched line whether an approval is needed, and move the Request to Pending if any Required line needs approval, otherwise to Allocated if every Required line is satisfiable and to Waiting if not; an Optional line's approvals never hold the Request in Pending.

Lifecycle.Hold.Place When a Request enters Pending the system shall place a Hold on every satisfiable Exclusive and Metered line. A Hold blocks other Requests exactly as an allocation does, posts no Charge, and its locked Quote counts as held credits so that concurrent Pending Requests cannot together exceed the Balance. A Hold commits when the Request becomes Allocated and otherwise releases when the Request leaves Pending or the Objection Window lapses; none is placed after the window has lapsed.

Lifecycle.Window.Start The Objection Window shall start when a Request is submitted with any line that needs approval, last the length set by the Capacity Planner (example default two working days), and not restart when a Candidate is replaced. On lapse with a Required approval outstanding, the system shall release every Hold, move the Request to Waiting, and escalate, within five minutes (PE-4).

Lifecycle.Allocate When every Required line is satisfied, the system shall in one transaction commit every Hold, post a Charge equal to each locked Quote, form the Loans and post the Earnings, drop Optional lines that are unsatisfiable or no longer affordable (telling the Manager which and why), leave Optional Matched lines still awaiting approval open, move the Request to Allocated, and notify the Manager; every other state change, Bump, waitlist fill, escalation, and approval request is notified in-app.

Lifecycle.Waiting.Reevaluate Whenever capacity a Waiting Request waits for frees, or an approval on it arrives, the system shall check whether every Required line can be satisfied at that moment; if so it allocates the Request at the locked Quotes, otherwise the freed capacity passes to the next waiting line and the Request's lines keep their positions. A replacement Candidate picked while the window still runs returns the Request to Pending with Holds re-placed.

4.9 Release and Refund

Description and priority. A Manager can Release all or part of an allocated Exclusive or Metered line for a Refund set by the Resource Type's Refund Schedule, a step function of lead time that never reaches 100%: holding capacity is never free, but an early Release beats a silent no-show. Ending a Loan early prorates the lending Team's Earning (FE-9; UC-9). **Priority:** High. **Stimulus/response.** The Manager Releases a line; the system posts the Refund, frees the capacity, and offers it to the waitlist.

Release.Line The system shall let the owning Team's Manager Release an allocated Exclusive or Metered line in whole or in part before it is consumed; a partial Release shortens the window, removes occurrences, or reduces units, the rest keeps its allocation, instance, and prices, and the freed capacity is offered to the waitlist at once (Exclusive.Waitlist.Join). Capacity neither used nor Released is never freed automatically and appears on the dashboard as idle.

Release.Refund On Release the system shall post one Refund equal to the Charge for the released part times

the fraction from the Refund Schedule for the lead time to the start of the released part's occurrence (example default 75% at seven or more days, 25% under seven days, 0% once that occurrence has started); no step may be 100% or more, and the Refund returns to the parts of the Account the Charge was drawn from.

Release.Loan.Prorate The borrowing Manager shall be able to end a Loan early; the system shall post to the lending Team an Earning correction so that its Earning equals the Day Rate times the days actually lent. No Refund Schedule applies. This correction is the only Transaction that may take a Balance below zero, and until a Grant or Earning restores it the Team can submit no Request with a Required Exclusive or Metered line.

4.10 Ledger and Pricing

Description and priority. The Ledger holds each Team's Account and every Transaction. Exclusive and Metered lines are charged credits, Matched lines nothing, and lending earns the Day Rate; granted credits expire with the Period while earned credits carry over. Prices vary by Price Band and lock into the Quote at submission; a demand surcharge is Stretch (FE-10; UC-11, UC-12). **Priority:** High. **Stimulus/response.** A Period begins; the system posts the Grants and expires the previous Period's granted credits.

Ledger.Account Each Team shall have one Account holding granted, earned, and held credits; Balance is granted plus earned, Available Balance is Balance minus held, and held credits equal the locked Quotes of the Team's lines on Hold. For each Team and Period the system shall produce a Statement with opening granted and earned credits, every Transaction, the amount expired, and closing balances; a Manager sees their own Team's, the Capacity Planner every Team's.

Ledger.Transaction.Kinds Every Transaction shall be a Grant, Charge, Earning, or Refund, recording Team, amount, Period, timestamp, actor or job, and the Request Line concerned; Transactions are never updated or deleted, and a correction is a new Transaction referencing the one it corrects.

Ledger.Grant.Period At the start of each Period the system shall post every Team's Grant as set by the Capacity Planner, who may also post a mid-Period Grant with a reason; at the end of a Period it shall set granted credits to zero, record the expired amount on the Statement, and leave earned credits untouched, within fifteen minutes of the boundary (PE-4). A Charge draws granted credits first, then earned, posts only when the Request becomes Allocated, equals the locked Quote, and never takes a Balance below zero.

Ledger.Price.Formula The Capacity Planner shall set a Unit Price per instance-hour or unit-hour for each Exclusive and Metered type and non-overlapping Price Bands (weekdays, time of day, multiplier; unbanded hours 1.0; example defaults weekdays 09:00–17:00 ×1.5, other hours ×0.5). A line's Quote is the sum over each hour of each occurrence of quantity times Unit Price times band multiplier, times the priority premium (example default 2×) for a Priority line.

Ledger.Earning.Post An Earning shall post to the lending Team's earned credits when a Loan forms, equal to the Day Rate times the Loan's person-days, a person-day being a Monday-to-Friday calendar day inside the window; the Day Rate is one value for every Person.

Ledger.Surcharge [Stretch] The Capacity Planner shall be able to enable a demand surcharge per Resource Type: an additional multiplier on the Unit Price that rises with the fraction of the type's capacity already allocated or held in the requested window at quote time, applied in Ledger.Price.Formula and locked with the Quote.

4.11 Showback Dashboard

Description and priority. The dashboard is showback: utilization and idle capacity per Resource Type, credits earned and spent per Team, and each Period's Statement (FE-11; UC-12). **Priority:** Medium. **Stimulus/response.** A Member opens the dashboard; the system shows the current Period's figures for their role, or any past Period they pick.

Dashboard.Utilization For each Exclusive and Metered Resource Type and Period the dashboard shall show utilization as a percentage (allocated instance-hours over available, or allocated unit-hours over Pool Capacity times hours), idle capacity (available minus allocated) for the Period to date and the coming seven days, and the number of lines waiting.

Dashboard.Credits For each Team and Period the dashboard shall show Grants, Charges, Earnings, Refunds, credits expired, Balance and Available Balance, and person-days lent and borrowed. A Manager sees company-wide utilization and only their own Team's credits and Loans; the Capacity Planner every Team; a Resource

Owner the Resources they own. Figures reflect every allocation, Release, and Transaction completed before the page was requested.

Dashboard.Usage [Stretch] The dashboard shall show requested versus used units per Team and Metered Resource Type when usage has been recorded (Metered.Usage.Compare).

4.12 Administration

Description and priority. An Administrator creates Members and Teams, grants roles, maintains the skill list, and defines Resource Types, and sets no policy; every Capacity Planner setting is specified here so that each has one home (FE-12; UC-1, UC-11). **Priority:** High. **Stimulus/response.** The Capacity Planner changes a setting; it applies to future Quotes, Grants, and windows and never alters a locked Quote or a posted Transaction.

Admin.Member.Create An Administrator shall be able to create and deactivate Members (name, email, username, initial password, Team), create, rename, and retire Teams, move a Member between Teams, maintain the skill list, and grant and revoke the roles Manager, Resource Owner, Capacity Planner, and Administrator; each Team has exactly one Manager at any time, and a deactivated Member cannot log in and appears on no Shortlist. Users authenticate by username and password (SE-2); roles are enforced on the server; Administrator screens contain no policy setting; every setting change is recorded with actor, old and new value, and timestamp.

Planner.Settings The Capacity Planner shall be able to set as data, without deployment: the Period (default one calendar month); each Team's Grant per Period; each Exclusive and Metered type's Unit Price, Price Bands, and Refund Schedule; the Day Rate; any Team's Quota on a Metered type; the priority premium; each Exclusive type's bump cutoff; and the Objection Window. A changed price affects only Quotes not yet locked; a changed window, cutoff, or schedule affects events after the change. The Capacity Planner shall see the escalated Requests and decide each outstanding approval from that list.

5 External Interface Requirements

5.1 User Interfaces

The system is a browser application. Each Member sees a menu tailored to their roles: Requests, Suggestions, Approvals, Inventory, Dashboard, Statement, Settings, and Administration. The signed-in Member and Team are always visible, along with a list of notifications and help available for each field (UD-3).

UI-1 The Draft Request screen shall show the lines as a table (Resource Type, class, quantity, window, Required or Optional, Priority mark where offered, Quote), the Required and Optional totals, the Available Balance and any shortfall, a panel of Open Suggestions, and the actions Save, Split line, Submit, Withdraw. The Request detail screen shall show the state, each line's state and locked Quote, and the state history.

UI-2 The Shortlist screen shall list Candidates in Shortlist order with name, Team, skills, and person-days lent in the current Period; the Approvals screen shall list approval requests with requesting Team, Person, window, Earning, and time left, with Approve and Refuse.

UI-3 The Inventory screens shall list Resource Types and Resources with filters and a timeline of Holds and allocations per Resource; the Dashboard and Statement screens shall show the figures of Section 4.11 with a Period selector; the Settings screen (Capacity Planner) shall show every setting of Planner.Settings and the escalated Requests; the Administration screens shall manage Members, Teams, roles, skills, and Resource Types.

5.2 Hardware Interfaces

None. The system manages records; it doesn't control doors, instruments, or compute nodes. Teams and people are responsible for acting on an allocation record.

5.3 Software Interfaces

All data is stored in PostgreSQL 16, a standard SQL database. Communication between the web interface and the back end goes only through a REST/JSON API over HTTPS. Every action described in the use cases can be done through this API with the same role controls. The back end is built in Java 17 with Spring Boot and runs in one Docker image. Release 1 doesn't connect to calendar, HR, identity, or cluster-scheduling systems (LI-3).

SI-1 All persistent data shall be stored in PostgreSQL 16 through JDBC with the schema migrated by the application at start-up; tests shall run against PostgreSQL 16 in a container started by the test suite (Testcontainers) with the same migrations.

SI-2 The front end shall talk to the back end only through a REST/JSON API over HTTPS, and every use case in Appendix D shall be achievable through that API with the same role enforcement; the back end runs on Java 17 with Spring Boot in one Docker image; Release 1 has no interface to any calendar, HR, identity, or cluster-scheduling system (LI-3).

5.4 Communications Interfaces

All network traffic between browser and server uses HTTPS. HTTP requests are redirected to HTTPS. In Release 1, all notifications are delivered inside the application. A stretch goal would be to send emails through an SMTP server that the Administrator sets up.

CI-1 All traffic between browser and server shall use HTTPS; plain HTTP shall only redirect to HTTPS.

CI-2 All notifications in Release 1 shall be delivered inside the application. [Stretch] Email delivery through an Administrator-configured SMTP server.

6 Other Nonfunctional Requirements

6.1 Performance Requirements

PE-1 The system shall support 500 Members, 50 Teams, 200 Resources, and 10,000 Requests per year without violating PE-2 or PE-3.

PE-2 Any page shall be served in at most 2 s at the 95th percentile with 50 concurrent users, at the scale of PE-1.

PE-3 Building a Shortlist against 500 Members shall take at most 3 s.

PE-4 Objection Window lapse, Hold expiry, and waitlist expiry shall take effect within 5 min of their due time; Period rollover within 15 min of the Period boundary.

6.2 Safety Requirements

None. The system manages records, not physical equipment; an allocation mistake is fixed with a Release or a Bump.

6.3 Security Requirements

The system keeps only the personal data it needs, checks roles on the server for every action, and shows each role only its own scope.

SE-1 The server shall enforce the signed-in Member's roles on every action, whether it arrives from the user interface or directly through the API.

SE-2 Passwords shall be stored only as salted hashes from an adaptive algorithm such as bcrypt; all communication shall be encrypted in transit (CI-1); sessions expire after 12 hours of inactivity and every authentication attempt is logged.

SE-3 The system shall store no personal data beyond name, email address, Team, skills, and availability.

SE-4 A Member shall see only what their roles allow: their own profile and Loans; as Manager, their Team's roster, Requests, Statements, and the Shortlists of their own Requests; as Capacity Planner, every Team's; as Resource Owner, the timelines of their Resources. Another Team's Statement is never visible to a Manager.

6.4 Software Quality Attributes

QA-1 **Correctness.** Under concurrent submissions the system shall never double-book an Exclusive instance for overlapping windows and never allocate a Metered type beyond its Pool Capacity in any hour; allocation is serialized per Resource Type in the database.

QA-2 **Atomicity.** Holds shall commit and the corresponding Charges and Earnings shall post in one transaction or not at all.

QA-3 **Auditability.** Ledger Transactions shall be append-only, and every Request and line state change shall record actor, change, and timestamp.

QA-4 **Availability and portability.** The system shall be available during business hours with planned maintenance outside them, with no high-availability requirement; the user interface shall work in the current release of Chrome, Firefox, Safari, and Edge, with no mobile layout.

QA-5 **Usability.** An untrained Manager shall be able to draft and submit a three-line Request in under 5 min, tested with at least 3 people from outside the team before the final demonstration.

QA-6 **Extensibility.** Adding a Resource Type shall require no code; adding an Allocation Class shall require implementing one allocator interface and no change to the existing allocators, the Request lifecycle, or the Ledger.

QA-7 **Data retention and testability.** Requests, state histories, Suggestions, and Transactions shall be kept for at least 2 years, nothing purged in Release 1; every requirement of Section 4 shall be covered by an automated test that runs against PostgreSQL in a container started by the test suite.

A Glossary

The vocabulary prefers the everyday word a Manager would use. Credit is the unit and never a transaction direction; there are no money words, because credits are showback, not billing. The glossary is generated at build time from the project's vocabulary file, which also lists the words each term deliberately replaces.

People and roles

Member Any user of the system. Belongs to exactly one Team, has a skill profile and availability, and is lendable as a Person.

Team The organizational unit that owns an Account and every Request made on its behalf. Has exactly one Manager.

Manager The Member who leads a Team. The only actor who submits Requests, and the approver for lending that Team's Members.

Resource Owner A Member responsible for the Inventory records of specific Exclusive or Metered resources. Never approves allocations.

Capacity Planner The role that sets each Team's Grant per Period, Unit Prices, the Day Rate, and Refund Schedules, and receives escalations.

Administrator The role that manages users, Teams, and Resource Type definitions. Configures the system; sets no policy.

Resources

Resource Anything that can be allocated: a Person, an Exclusive resource such as a room, or a Metered pool such as GPU hours. Every Resource has exactly one Resource Type.

Resource Type A named definition, created as data by an Administrator, that fixes a Resource's Allocation Class and attributes.

Allocation Class One of three policies a Resource Type declares, deciding how Requests for it are allocated: Matched, Exclusive, or Metered.

Matched The Allocation Class for People. Allocated by matching skills and availability, gated by the Person's Manager.

Person A Member viewed as a Matched Resource: someone with skills and availability who can be lent to another Team.

Exclusive The Allocation Class for Resources held whole over a time window, such as rooms or instruments. Allocated with conflict detection, a waitlist, and priority bumping.

Metered The Allocation Class for divisible Resources drawn against a Team quota, such as compute hours or license seats.

Pool Capacity The total units a Metered Resource Type can supply at once, such as 32 GPUs, maintained by its Resource Owner. Allocations in any hour may not exceed it.

Quota An optional per-Team cap on the units of one Metered Resource Type the Team may hold concurrently, set by the Capacity Planner. A safety valve against monopolizing a pool, not a rationing mechanism.

Inventory The record of every Resource that exists, its Resource Type, and who currently holds it or has it on Hold.

Requests

Request A single request, owned by a Team and submitted by its Manager, containing one or more Request Lines that may span all three Allocation Classes.

Request Line One need inside a Request: a Resource Type, a quantity, and a time window. Every line has exactly one Allocation Class, taken from its Resource Type.

Suggestion A single Request Line a Member proposes to their own Manager. Has no allocation effect until the Manager accepts it into a Request; several Suggestions may be accepted into the same line.

Required line / Optional line A Manager's marking on each Request Line. A Request is fulfilled only when every Required line is allocated; Optional lines allocate if they can and never block the outcome.

Hold A tentative, uncharged allocation of an Exclusive or Metered line placed while a Request's Matched lines await approval. Commits when the Request is fulfilled, releases when it fails or the Objection Window lapses.

Objection Window The bounded period a lending Manager has to approve or refuse a Matched line before the Request escalates to the Capacity Planner. Also the lifetime of any Hold on that Request.

Candidate A Person who satisfies a Matched line's skills and availability, has no Loan overlapping the line's window, and appears on its Shortlist. Becomes a Loan once their Manager approves.

Shortlist The list of Candidates the system produces for a skill-based Matched line, ordered by fewest person-days lent in the current Period, from which the requesting Manager selects.

Waitlist The ordered list of Required lines that could not be satisfied for a Resource Type and window. Priority lines first, then by submission time; a Bumped line goes to the head. Optional lines are never waitlisted.

Loan A Person allocated to a Team other than their own for a time window. The fulfilled form of a Matched line. A Person is in at most one Loan at a time.

Priority line An Exclusive Request Line that is charged a premium, a multiple of the Unit Price set by the Capacity Planner. Can Bump a standard line and cannot itself be Bumped.

Bump The displacement of a standard Exclusive line by a Priority line. The displaced line goes to the head of the waitlist with a full Refund. Not allowed inside the bump cutoff before the window starts.

Release A Manager giving back an allocated Exclusive or Metered line, in whole or in part, before it is consumed, or ending a Loan early. Triggers a Refund according to the Refund Schedule.

Credits

Credit The unit of account in the Ledger. No real money; used for showback, not billing.

Ledger The record of every Team's Account and every Transaction. Holds no capacity information.

Account A Team's standing in the Ledger: its granted credits, its earned credits, and its held credits.

Balance A Team's granted credits plus earned credits.

Available Balance Balance minus held credits. What a Team can commit to a new Request.

Held credits Credits set aside for a Request that is Pending, equal to what its Holds will cost when they commit.

Transaction One entry in the Ledger against an Account. Exactly four kinds: a Grant (from the Capacity Planner), a Charge (for an allocated Exclusive or Metered line), an Earning (for a Loan), or a Refund (for a Release).

Grant Credits given to a Team by the Capacity Planner at the start of a Period, or as a mid-period adjustment. Expire at the end of the Period.

Period The fixed calendar interval over which Grants are issued and expire. Set by the Capacity Planner.

Statement A Team's per-Period view of its Account: Grants, Charges, Earnings, Refunds, and what expired. The showback artifact.

Unit Price The base credits per unit of an Exclusive or Metered Resource Type (per hour of a room, per GPU hour), set by the Capacity Planner, before any Price Band multiplier. Matched lines have no Unit Price.

Price Band A time-of-day and day-of-week window, defined per Resource Type by the Capacity Planner, carrying a multiplier on the Unit Price. Rewards use at low-demand times.

Quote The credits a Request Line will cost, shown to the Manager while drafting and locked when the Request is submitted. The Charge that posts equals the Quote.

Day Rate The flat credits a Team earns per person-day of a Loan, set once by the Capacity Planner. The same for every Person regardless of skill.

Refund Schedule The rule, set per Resource Type by the Capacity Planner, giving the fraction of a Charge returned on Release as a step function of how far ahead of the window start the Release happens. Never reaches 100%.

B Data Dictionary and Data Model

Figure 2 shows the entities the requirements name. The Inventory is the Resource Type and Resource entities with the Holds and allocations on them; the Ledger is the Account and Transaction entities; neither holds the other's information. The full specification carries the complete data dictionary with every attribute's type; the elements below are the ones the requirements depend on most.

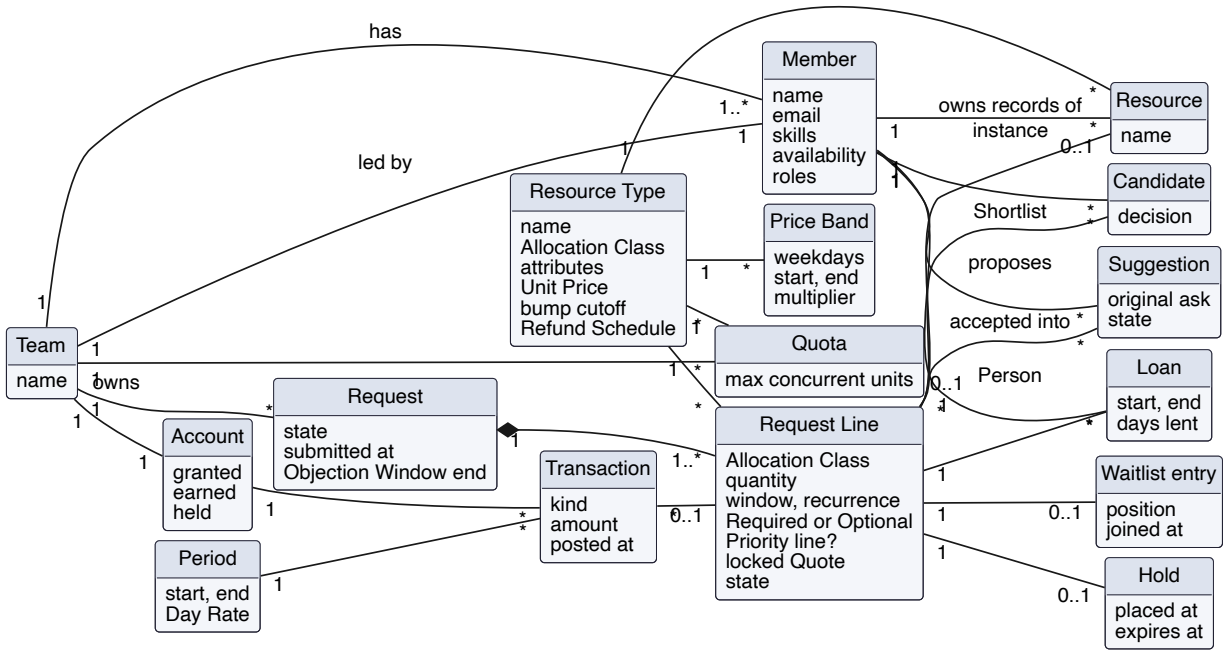


Figure 2: Data model. Multiplicities read along each line; a Statement is a per-Period view over Transactions and is not stored.

Element	Meaning	Composition
Resource Type	Data that fixes an Allocation Class and attributes.	name; Allocation Class; attribute definitions; Unit Price; Price Bands; bump cutoff; Refund Schedule
Resource	One thing that can be allocated; a Metered Resource is a pool.	name; Resource Type; attribute values; Resource Owner; units (Metered)
Team	The organizational unit that owns an Account and its Requests.	name; Manager (one Member); retired
Member	Any user; belongs to one Team; lendable as a Person.	name; email; username; password hash; Team; skills (from the skill list); availability (date ranges); roles; active
Request	One request of a Team, submitted by its Manager.	Team; state; submitted at; Objection Window end; lines; state history (actor, change, timestamp)
Request Line	One need inside a Request.	Resource Type; quantity; window; recurrence; Required or Optional; Priority line; skills or named Person; locked Quote; allocated instance; state: Open, Awaiting approval, Approved, Held, Waitlisted, Allocated, Dropped, Released, Bumped, Expired
Suggestion	A single proposed line from a Member to their Manager.	Member; Manager; proposed line (same fields as a Request Line); note; state: Open, Accepted, Declined; accepted into (Request Line); decline reason
Candidate	A Person on a Shortlist, and the approval asked of their Manager.	Request Line; Member; person-days lent in the current Period; selected; decision: none, Approved, Refused; decided by; decided at; reason
Hold	A tentative, uncharged allocation.	Request Line; Resource (Exclusive) or units (Metered); placed at; expires at (Objection Window end)
Loan	A Person allocated to another Team for a window.	Request Line; Member; lending Team; borrowing Team; start; end (may be shortened); person-days (derived)
Waitlist entry	A Required line waiting for a Resource Type and window.	Request Line; Resource Type; window; position (derived from Priority, Bump, submission time); joined at
Account	A Team's standing in the Ledger.	granted; earned; held (derived from Holds); Balance and Available Balance (derived)
Transaction	One append-only Ledger entry.	kind (Grant, Charge, Earning, Refund); amount; drawn from granted and earned; Period; posted at; actor; Request Line; corrects
Period	The interval over which Grants are issued and expire.	start; end; Grant per Team; Day Rate in force

C Analysis Models

Figure 3 shows the states of a Request and every transition; Section 4.8 gives the guards. Three properties carry the design: a Hold exists only while the Request is Pending, so none outlives the Objection Window; Charges and Earnings post only on entry to Allocated; and Waiting is the one state from which a Request recovers after a refusal, a lapse, or a Bump. A typical three-class Request runs Draft, Submitted, Pending (room and GPUs on Hold, people awaiting approval), Allocated.

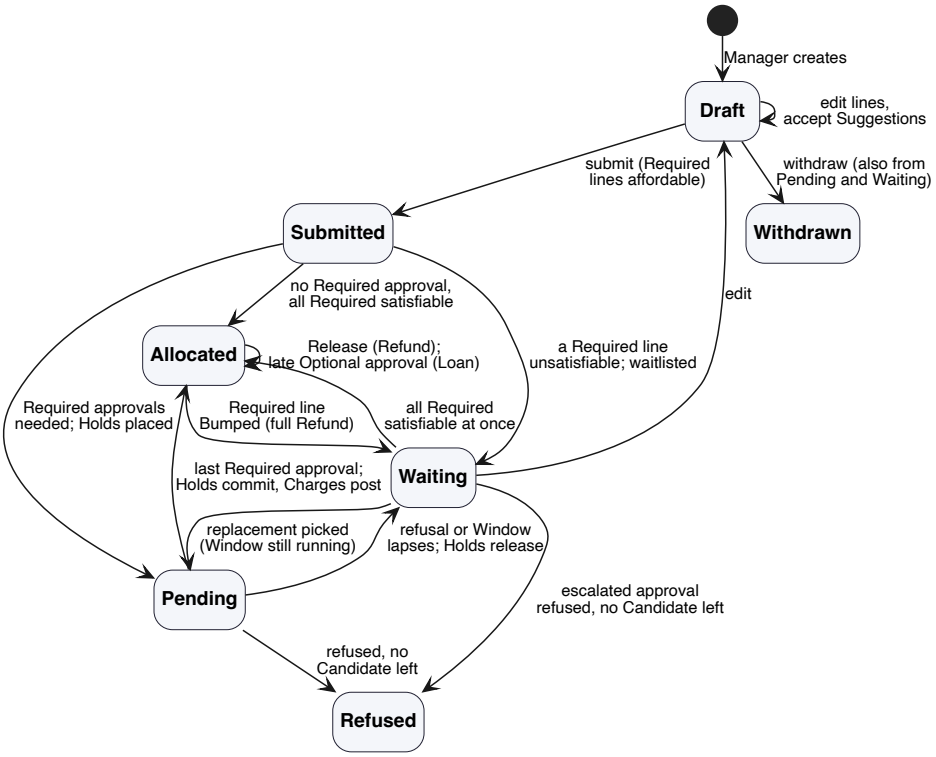


Figure 3: Request state-transition diagram. Submitted is a transient evaluation state. Withdraw is drawn once, from Draft; a Manager may also withdraw from Pending and Waiting. Allocated, Withdrawn, and Refused are final, except that Allocated returns to Waiting when a Required line is Bumped.

D Use Cases

Figure 4 shows the twelve use cases and the actors that start them; the Clock stands for the scheduled jobs that fire when an Objection Window lapses or a Hold expires. Table 2 gives each one's primary actor, trigger, and outcome. The full specification writes each use case out in the fields of the template: description, preconditions, postconditions, normal, alternative, and exception flows, priority, and frequency.

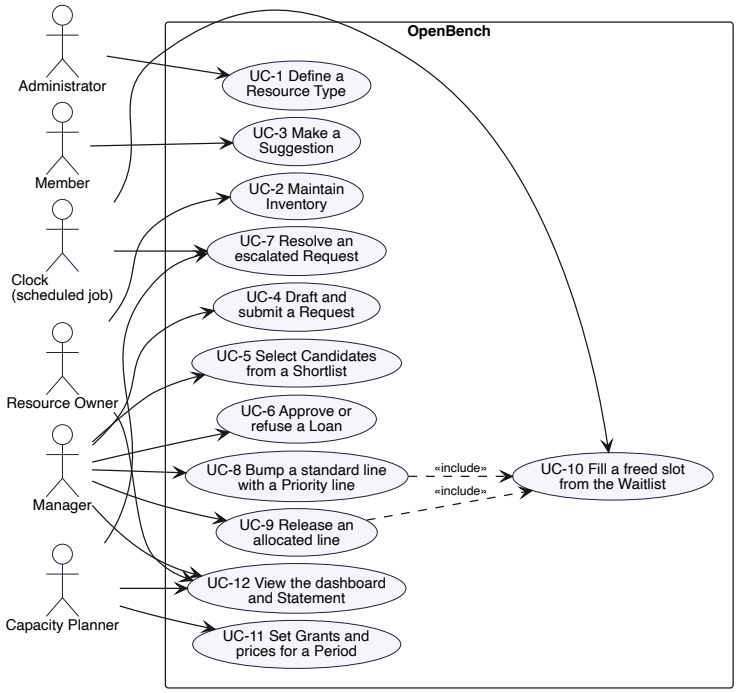


Figure 4: Use case diagram. All human actors are Members; the roles are additive.

Table 2: The twelve use cases.

ID	Name	Primary actor	Trigger and outcome
UC-1	Define a Resource Type	Administrator	A new kind of Resource is offered; the type exists as data and lines can name it.
UC-2	Maintain Inventory	Resource Owner	A Resource enters or leaves service or changes units, never below what is allocated or held.
UC-3	Make a Suggestion	Member	A Member proposes one line; the Manager accepts it into a Draft or declines it.
UC-4	Draft and submit a Request	Manager	The Request becomes Pending with Holds, Allocated, or Waiting, every Quote locked.
UC-5	Select Candidates from a Shortlist	Manager (requesting)	A Request with a skill-based line is submitted; approval requests go to the chosen Candidates' Managers.
UC-6	Approve or refuse a Loan	Manager (lending)	Approved (possibly allocating the Request) or refused (Holds release, replacement asked).
UC-7	Resolve an escalated Request	Capacity Planner	The Objection Window lapses; Holds release and the Capacity Planner decides the approvals.
UC-8	Bump a standard line with a Priority line	Manager	A Priority line finds no free instance; outside the cutoff the blocking line is refunded and moved to the head of the waitlist.
UC-9	Release an allocated line	Manager	Capacity is freed, a partial Refund posts, and the waitlist is offered the slot.
UC-10	Fill a freed slot from the Waitlist	Clock, system events	Freed capacity satisfies the first fitting waiting line, allocating or holding its Request.
UC-11	Set Grants and prices for a Period	Capacity Planner	Settings are saved; at the boundary granted credits expire and new Grants post.
UC-12	View the dashboard and Statement	Manager, Capacity Planner, Resource Owner	Utilization, idle capacity, credits, and the Statement are shown for the viewer's scope.

E Inspection Report

One row per finding, filled in by the inspectors. Categories: ambiguity, incompleteness, incorrectness, inconsistency, infeasibility, traceability, clarity; status Open, Closed (version), or Rejected (reason).

All four team members, J. Nana, B. Lujan, F. Turopov, and Y. Shnazbayev, each read the whole document and took part in the inspection meeting on 2026-09-21, at which the findings were categorized and resolved.

ID	Found by	Location	Category	Finding	Resolution	Status
I-1	F. Turopov	Sections 1–3	clarity	Dense sentences written for the authors, not a first-time reader; no way into the vocabulary before the glossary.	Reworded; Definitions paragraph added to Section 2; scope content unchanged.	Closed (0.2)
I-2	B. Lujan	Sections 4–6	clarity	Feature descriptions restated the requirements in compound sentences; Sections 5 and 6 opened with bare requirement lists.	Descriptions of 4.1–4.12 rewritten and condensed; lead-ins added to Sections 5 and 6; shall statements unchanged.	Closed (0.2)
I-3	Y. Shnazbayev	Whole document	clarity	Sizes and spacing varied between sections and tables; sections ran on from one another; some section files were shared with the full edition.	One body size, 8pt tables, 3pt item gaps, compact headings; every section on a new page (F with E); own copy of every section.	Closed (0.2)

F Traceability

Each feature maps to its Section 4 subsection, whose requirements share its identifier prefix, and to the use cases of Appendix D that exercise it.

Feature (section)	Use cases
FE-1 Inventory (4.1)	UC-1, UC-2
FE-2 Profiles (4.2)	UC-5
FE-3 Suggestions (4.3)	UC-3, UC-4
FE-4 Draft and Submit (4.4)	UC-4, UC-8
FE-5 Matched (4.5)	UC-5, UC-6, UC-7
FE-6 Exclusive (4.6)	UC-4, UC-8, UC-10
FE-7 Metered (4.7)	UC-4, UC-10
FE-8 Lifecycle (4.8)	UC-4, UC-6, UC-7, UC-10
FE-9 Release (4.9)	UC-9, UC-10
FE-10 Ledger (4.10)	UC-11, UC-12
FE-11 Dashboard (4.11)	UC-12
FE-12 Administration (4.12)	UC-1, UC-11, UC-7